Exploratory data analysis (EDA)

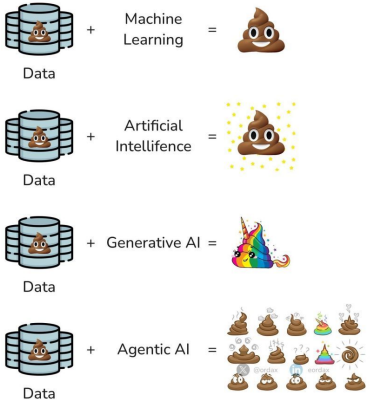
Material based on Chapter 3 of "The Orange Book of Machine Learning - Green edition" by Carl McBride Ellis

Carl McBride Ellis

carl.mcbride@u-tad.com

Why perform exploratory data analysis?

"Garbage in, garbage out" (GIGO)



> Many people assume that most data quality issues are caused by data entry errors. A more sophisticated understanding recognizes that gaps in or poor execution of business and technical processes cause many more problems than mis-keying. However, common sense says and research indicates that many data quality problems are caused by a lack of organizational commitment to high quality data, which itself stems from a lack of leadership, in the form of both governance and management.

(Source: "DAMA-DMBOK2: Data Management Body of Knowledge")

See also:

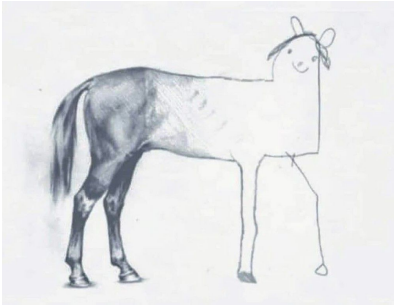
- ISO/IEC 25012:2008 general data quality model
- ISO/IEC 25024:2015 measurement of data quality

Data Assessment Matrix

	Insufficient	Developing	Functional	Optimal
Appropriateness	Data unrelated to the research question(s).	Target variables are inappropriate for the research question.	Features aren't entirely appropriate for the research question.	Clear relation between features and target variables.
Readiness	No data collected or collection methodology.	Some collected data or clear collection methodology in place.	Data collected, but requires some merging and unifying.	Data collected and cleaned.
Reliability	Strongly biased data.	Indeterminate source of bias.	Determinate source of bias which can be accounted for.	Weak or clear of bias.
Sensitivity	Highly risky sensitive data.	Requires extensive security measures.	Data does not present significant risk but may be confidential.	Limited risks associated with the data, usually publicly available data.
Sufficiency	Prior experience shows low success rate.	Does not meet the level of previous succesful events.	Previous events have successfully operated at this level.	Extensive data in excess of previous successful events.

(Fuente: "*How to Data in Datathons*")

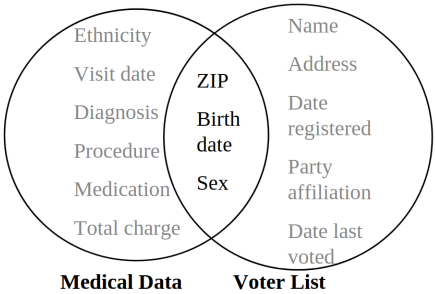
- 1. Appropriateness: is the data actually relevant to the data question(s) posed?
- 2. Sufficiency: how much data is enough data?



3. Readiness: is the data analysis-ready?



4. Sensitivity: how sensitive is the data? Does it contain personal information or data which carries legal, reputational, or political risks?



With just three features in a dataset 87% of the US population can be uniquely identified

(Source: *"Simple Demographics Often Identify People Uniquely"*)

5. Reliability: how biased is the data? Biases include historical, representation, measurement and aggregation.



Chicago Daily Tribune

THE WORLD'S GREATEST NEWSPAPER

HOME

WEDNESDAY, NOVEMBER 2, 1948

DEWEY DEFEATS TRUMAN

G.O.P. Sweep Indicated in State; Boyle Leads in City

REPUBLICAN TICKET AHEAD OF 1944 VOTE

Tops Coghlan in Hot Race for Attorney

Probable Winners

Town Balloting Gives Trend

Probable Winners

RECORD CITY VOTE SEEN IN LATE TALLIES

Suburban Ballot Near 375,000

BULLETINS ON ELECTIONS

Early Count Gives G.O.P. Senate Edge

PUTS G.O.P. BACK IN THE WHITE HOUSE

Sizable Electoral Margin Seen

Getting to know your dataframe (`df`):

- `df.shape` - the number of rows and columnas in the `df`
- `df.head(n)` - print the first n rows
- `df.tail(n)` - print the last n rows
- `df.dtypes` - show the column types (float, integer, object, *etc.*)
- `df.describe()` - some descriptive statistics for each column

Descriptive statistics ✔
Example:

	count	mean	std	min	25%	50%	75%	max
PassengerId	891.000000	446.000000	257.353842	1.000000	223.500000	446.000000	668.500000	891.000000
Survived	891.000000	0.383838	0.486592	0.000000	0.000000	0.000000	1.000000	1.000000
Pclass	891.000000	2.308642	0.836071	1.000000	2.000000	3.000000	3.000000	3.000000
Age	714.000000	29.699118	14.526497	0.420000	20.125000	28.000000	38.000000	80.000000
SibSp	891.000000	0.523008	1.102743	0.000000	0.000000	0.000000	1.000000	8.000000
Parch	891.000000	0.381594	0.806057	0.000000	0.000000	0.000000	0.000000	6.000000
Fare	891.000000	32.204208	49.693429	0.000000	7.910400	14.454200	31.000000	512.329200

obtained from:

```
df.describe().T
```

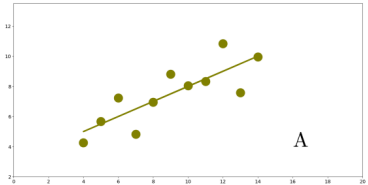
EDA: how and why?

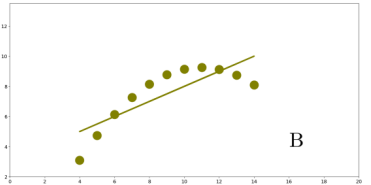
- Anscombe's quintet
- **Univariate visualisation**
  - box and violin plots ✗
  - histograms ✓
  - anomalous values, outliers and inliers
  - normality, skewness and kurtosis
  - categorical features
- **Bivariate visualisation**
  - correlación (Pearson)
  - scatter plots ✓
- Why: to help with feature selection and engineering - ideas

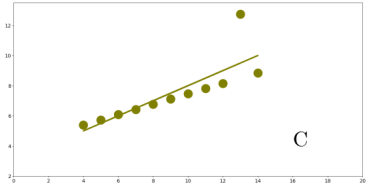
Vizualizing data: "*Anscombe's quartet*" (1973)

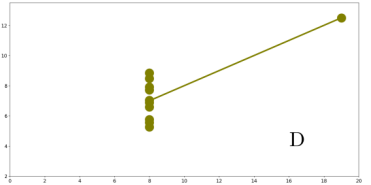
Statistic	A	B	C	D	E
$\bar{x}$	9.0	9.0	9.0	9.0	9.0
$\bar{y}$	7.5	7.5	7.5	7.5	7.5
$\sigma(x)$	3.317	3.317	3.317	3.317	3.317
$\sigma(y)$	2.032	2.032	2.030	2.031	2.031
β_1	0.5	0.5	0.5	0.5	0.5
β_0	3.0	3.0	3.0	3.0	3.0
$\sum(x-\bar{x})^2$	110.0	110.0	110.0	110.0	110.0
RSS	13.76	13.78	13.76	13.74	13.76
R^2	0.667	0.666	0.666	0.667	0.666
regression sum of squares	27.51	27.50	27.47	27.49	27.48
standard error of β_1	0.118	0.118	0.118	0.118	0.118
Pearson r_{xy}	0.816	0.816	0.816	0.817	0.816

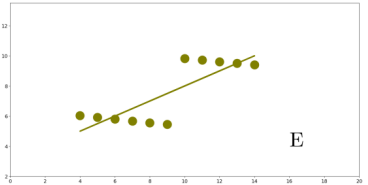
(Notebook: Anscombe's quartet and the importance of EDA)


A


B


C


D


E

Anscombe's quintet:
We have five datasets that have the very same descriptive statistics, but in reality they are very different:

- A. we have a good fit with a straight line
- B. underfitting
- C. we have an 'outlier' point
- D. we have a levarage point
- E. an example of Simpson's paradox

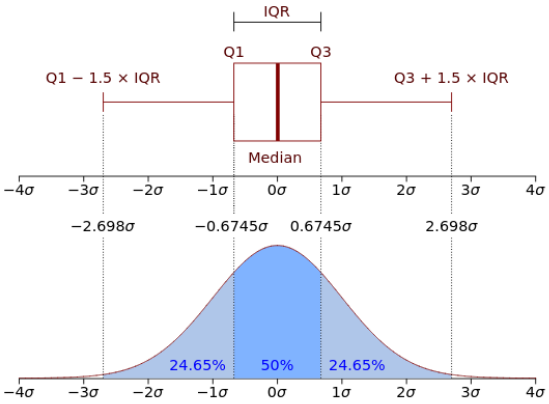
Message: we have to visualize our data; calculations are not enough

(Modern version: *The Datasaurus Dozen*)


EDA_DataSaurus_Dozen.gif

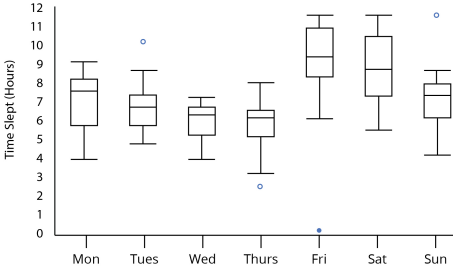
Univariate (1D) visualisation

Interquartile range (IQR): The range between 25% and 75% of the datapoints



note however that our data are not (basically never) necessarily Gaussian

Box plots ✖✖✖

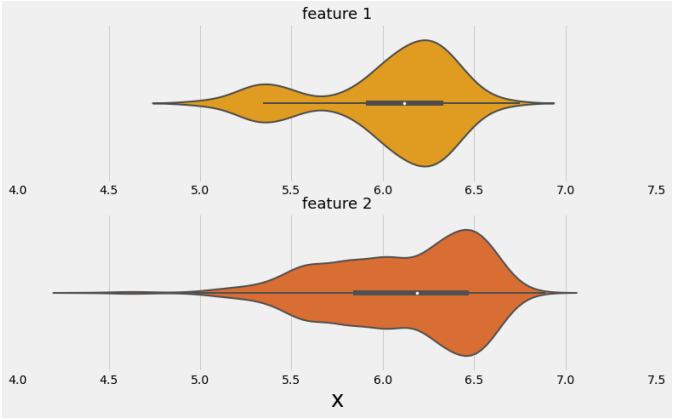


Notebook: Box + strip + violin = raincloud plot

Problem: 🎥 `EDA_box_plot_fail.gif`, `EDA_box_plot_animation.gif`

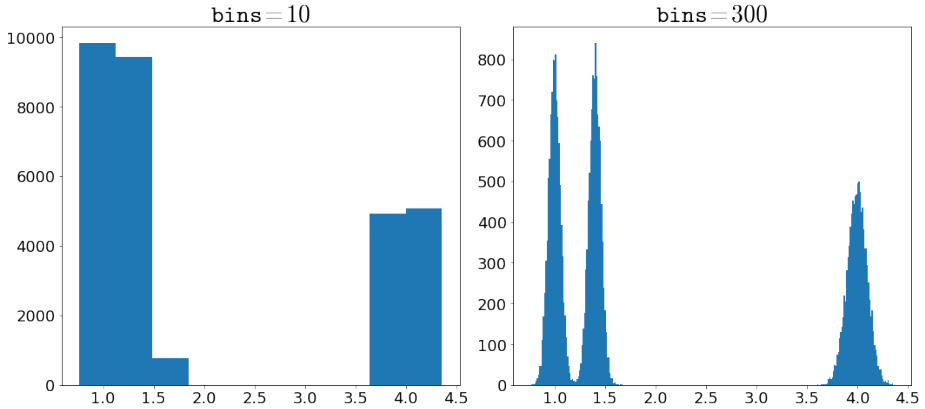
(Nick Desbarats *"I've Stopped Using Box Plots. Should You?"*)

violin plots ✖

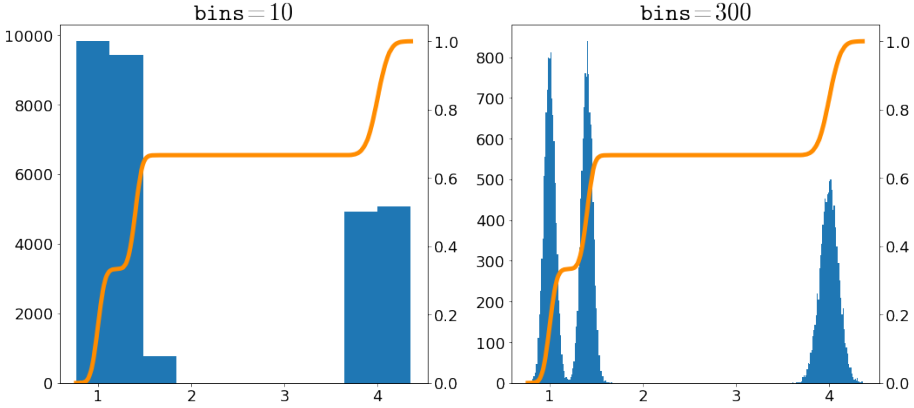


Histograms ✔✔✔

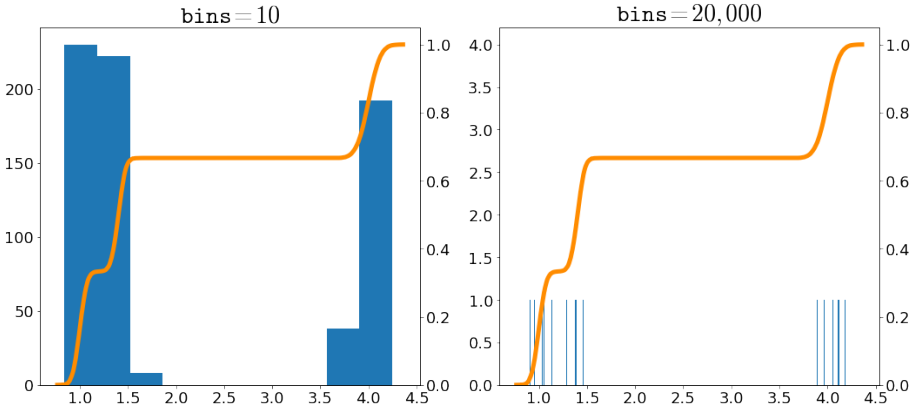
Watch out when choosing the number/width of the *bins*! These are two histograms made using the **very same data**:



Solution: the *empirical cumulative distribution function* (eCDF)
The *eCDF* is independent of the number of bins
(*i.e.* in `seaborn` use `cumulative=True` or `seaborn.ecdfplot`)



...and if there are too many bins:



...we basically end up with what is known as a 1-dimensional rug plot.

how to create a histogram + eCDF ✔✔✔

```
import seaborn as sns

fig, (ax1) = plt.subplots(1, 1, figsize=(15, 5))
ax2 = ax1.twinx()
sns.histplot(data=df, x="feature", bins=??, ax=ax1)
sns.ecdfplot(data=df, x="feature", color="red", lw=2,  ax=ax2)
plt.show();
```

Outliers

Atypical 'outlier' values:

Note: It is important to differenciate between extreme points and bad points

possible causes:

- human error
- measurement error
- ...

for example

- the data shows person who is 6 meters tall: someone confused the units of feet and meters
- a temperature reading of -150 degrees in the Sahara: a thermometer has broken

Outlier detection ✖

- $\geq \pm 1.5$ IQR
- z-score $= \frac{X - \bar{x}}{\sigma}$ = `stats.zscore()`

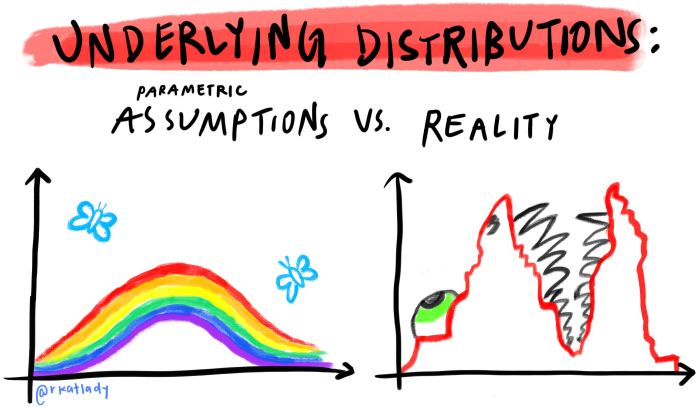
for example:

```
from scipy.stats import zscore
df_z = df.apply(zscore)
```

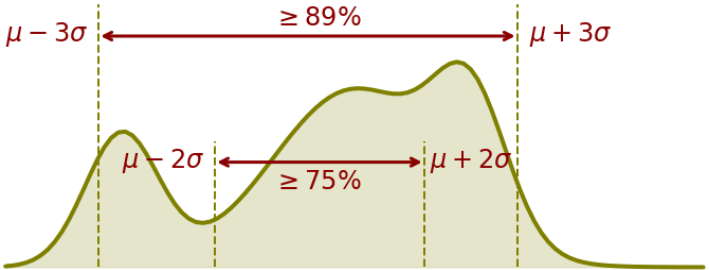
values $> |3|$ are fruther from μ than 99.7% of the rest of the data

(Note: The z-score cannot detect 'inliers') (see also the distancia de Mahalanobis)

However:

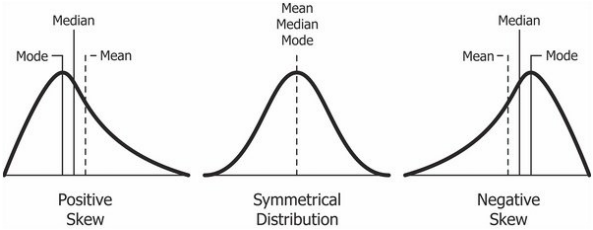

UNDERLYING DISTRIBUTIONS:
PARAMETRIC
ASSUMPTIONS VS. REALITY
@rkatlady

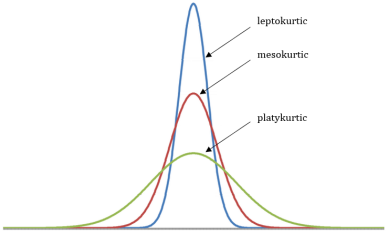
Chebyshev's inequality: in general $\pm 3\sigma$ is only 89% of the data




±3 SD
bad data scientist
imgflip.com

Use EDA to see if the features have skewness o kurtosis





This can be 'corrected' using a Yeo-Johnson transform (scikit `power_transform`)

```
from scipy.stats import skew
print( skew( df["feature"] ) )

from scipy.stats import kurtosis
print( kurtosis( df["feature"] ) )
```

Both skew and kurtosis will be zero for a Gaussian distribution

Normality tests (null hypothesis (H_0): the data follows a normal distribution)

- Prueba de Shapiro-Wilk
- D'Agostino test
- Anderson-Darling test

(Paper: *"Comparisons of various types of normality tests"*)

Note that when we have skewed distributions, the mean is no longer a good measure of central tendency, and the median is more robust.

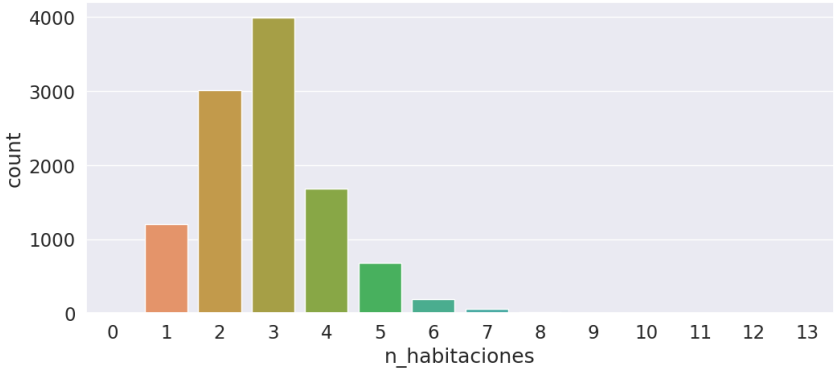
Neither is the
standard deviation a robust measure, and it is better to use the median absolute deviation (MAD)

$$\text{MAD}(y) := \text{median}(|y_i - \tilde{y}|)$$

where $\tilde{y}$ is the median.

Categorical features

countplot ✔



how to make a countplot

```
import seaborn as sns

fig, (ax1) = plt.subplots(1, 1, figsize=(12, 5))
sns.countplot(x=df["feature"]);
```

pandas groupby

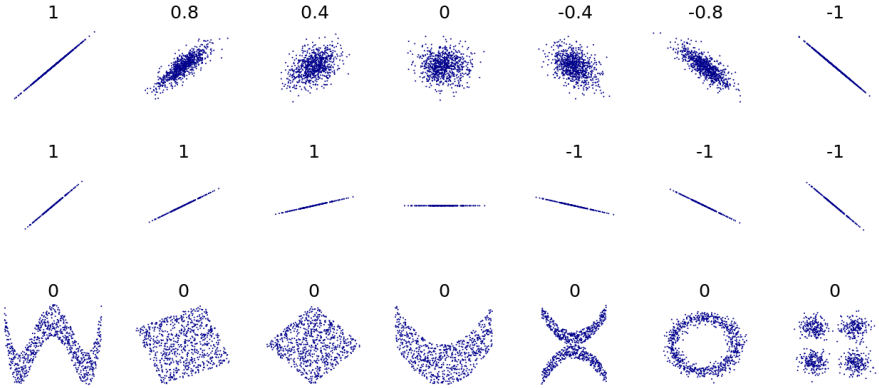
```
df.groupby(by=["cat_col"])[['target']].count()
df.groupby(by=["cat_col"])[['target']].mean()
df.groupby(by=["cat_col"])[['target']].std()
df.groupby(by=["cat_col"])[['target']].min()
df.groupby(by=["cat_col"])[['target']].quantile(q=0.25)
df.groupby(by=["cat_col"])[['target']].median()
df.groupby(by=["cat_col"])[['target']].quantile(q=0.75)
df.groupby(by=["cat_col"])[['target']].max()
```

Bivariate (2D) visualisation

The Pearson correlation coefficient for linear relations

$$r_{xy} := \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}} = \frac{cov(x, y)}{\sigma_x \sigma_y} \qquad \in [-1, 1]$$

where cov is the covariance and σ is the standard deviation. (Remember that correlation does not imply causation)



(ver tb.: "Pearson correlation coefficient, mutual information (MI) and Predictive Power Score (PPS)")

Correlation matrix ✔

	date	weight	resp_1	resp_2	resp_3	resp_4	resp	feature_0	feature_1	feature_2
date	1.00	0.02	0.02	0.01	0.01	-0.02	-0.03	0.01	-0.05	-0.01
weight	0.02	1.00	-0.01	-0.01	-0.02	-0.02	-0.02	-0.02	-0.05	0.06
resp_1	0.02	-0.01	1.00	0.92	0.80	0.49	0.58	0.04	0.03	0.02
resp_2	0.01	-0.01	0.92	1.00	0.91	0.55	0.68	0.04	0.04	0.01
resp_3	0.01	-0.02	0.80	0.91	1.00	0.77	0.80	0.03	0.04	0.01
resp_4	-0.02	-0.02	0.49	0.55	0.77	1.00	0.95	-0.06	0.00	-0.04
resp	-0.03	-0.02	0.58	0.68	0.80	0.95	1.00	-0.06	0.00	-0.04
feature_0	0.01	-0.02	0.04	0.04	0.03	-0.06	-0.06	1.00	0.03	0.03
feature_1	-0.05	-0.05	0.03	0.04	0.04	0.00	0.00	0.03	1.00	0.84
feature_2	-0.01	0.06	0.02	0.01	0.01	-0.04	-0.04	0.03	0.84	1.00

How to calculate a correlation matrix

```
df.corr(method='pearson', numeric_only=True) \
  .style.background_gradient(cmap='coolwarm', vmin=-1, vmax=1) \
  .format(precision=2)
```

and in the form of a 'ranking' ✔✔✔

```
import pandas as pd
import itertools

columns_list = df.select_dtypes(include='number').columns.tolist()
pairs = itertools.combinations(columns_list, 2)
correlation_list = []

for pair in (pairs):
    correlation_list.append([pair[0], pair[1],
                             df[pair[0]].corr(df[pair[1]])])

correlation_df = pd.DataFrame(correlation_list)
# display the results
correlation_df.sort_values(by=[2], ascending=False)
```

Highly correlated columns are usually bad to have, they make the interpretation of a fit more difficult.

It is also possible that highly correlated columns could represent the very same thing (multicollinearity) For example, speed in both km/h and mph.

It is convenient to delete (*drop*) one of them:

```
df = df.drop(["kmh"], axis=1)
```

(see also: Explainability, collinearity and the variance inflation factor (VIF))

Why is multicollinearity a problem for linear regression?

Here the features 🍏 and 🍇 are perfectly correlated:

🍏	🍇	total
1🍏	2🍇	7€
10🍏	20🍇	70€
100🍏	200🍇	700€

Regression: spurious correlation

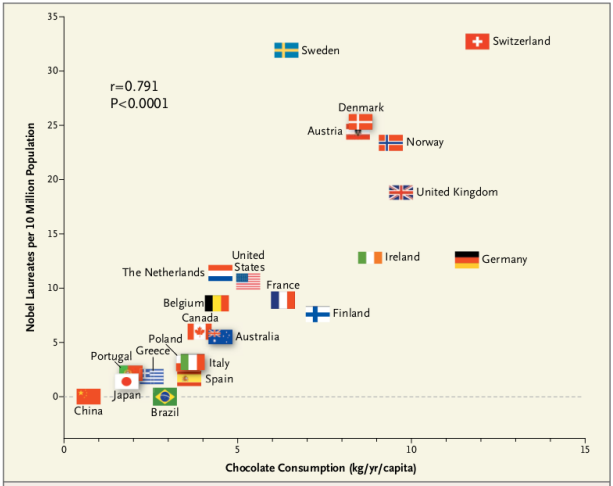
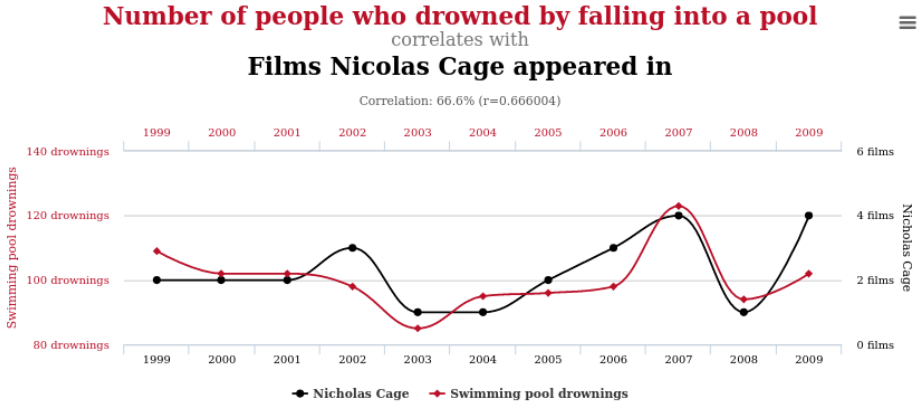


Figure 1. Correlation between Countries' Annual Per Capita Chocolate Consumption and the Number of Nobel Laureates per 10 Million Population.

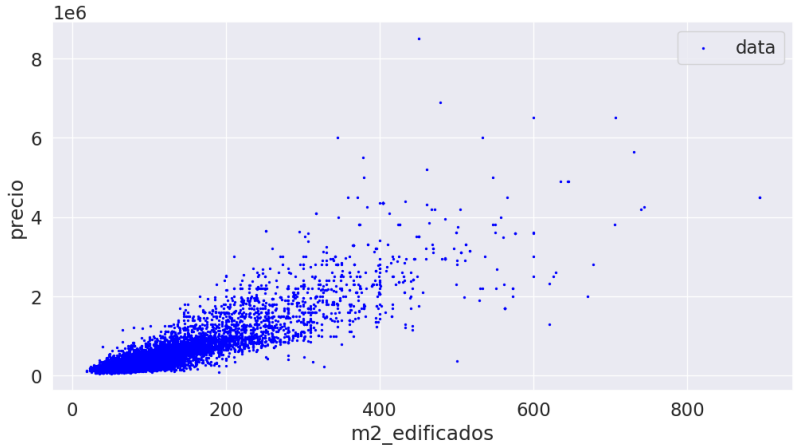
(Fuente: "*Chocolate consumption, cognitive function, and Nobel laureates*" Franz H. Messerli, **N. Engl. J. Med.** (2012))

...and for time series



(fuente: la pagina web Spurious Correlations)

Scatter plot ✔✔✔



how to draw a scatter plot

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(12, 6))
ax.scatter(x=df["feature_1"], y=df["feature_2"],
    color = 'blue',
    label = "data",
    s=2 # change point size
    )
# como cambiar los limites del plot:
#ax.set(xlim=(0, 100), ylim=(0, 100))
ax.set_xlabel ("feature_1")
ax.set_ylabel ("feature_2")
plt.legend(loc="upper right")
plt.show();
```

or to create an interactive scatter plot using plotly

```
import plotly.express as px

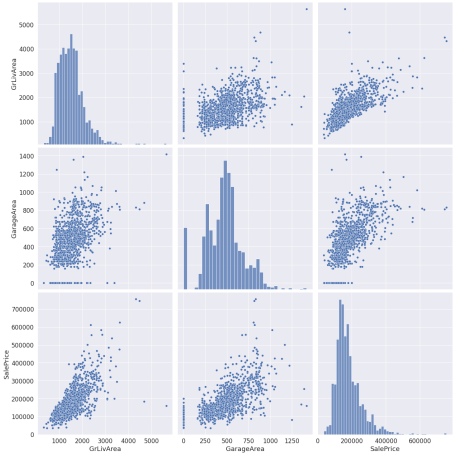
fig = px.scatter(x=df["feature_1"], y=df["feature_2"])
fig.show()
```

also: how to draw an interactive 3D scatter plot

```
import plotly.express as px
import plotly.graph_objects as go

fig = go.Figure()
fig = px.scatter_3d(
    x=df["feature_1"],
    y=df["feature_2"],
    z=df["feature_3"],
    title="Mi 3D plot",
    labels={"x": "feature_1",
            "y": "feature_2",
            "z": "feature_3"}
    )
fig.update_traces(marker=dict(size=1.5)) # change point size
fig.show()
```

Pairplot ✖



function: `sns.pairplot(df, height=3)`

(disadvantage: if there are n features you will have n^2 plots)

alternativa (source: gist)

```
import matplotlib.pyplot as plt
import seaborn as sns
import itertools

df = dataset.select_dtypes(include='number')
numeric_features = df.columns.tolist()
pairs = itertools.combinations(numeric_features, 2)

for pair in (pairs):
    sns.lmplot(data=df, x=pair[0], y=pair[1],
               line_kws={'color': 'black'},
               aspect=1 )
    plt.show();
```

Note: this will produce $(n^2 - n)/2$ plots, where n is the number of features

Data vizualization: the three main python libraries

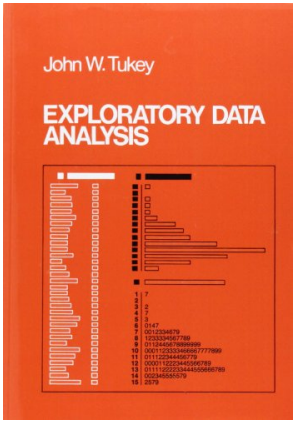
- matplotlib – `import matplotlib.pyplot as plt`
- seaborn – `import seaborn as sns`
- plotly express (interactivo) – `import plotly.express as px`

Galleries with corresponding source code:

- matplotlib gallery
- seaborn gallery
- plotly express gallery

(Ver tb.: *PyViz: libraries for visualizing data in Python*)

Classic text from 1977 by John W. Tukey:



(BTW: John Tukey also invented the fast Fourier transform (FFT) algorithm)

Practical session I

Use EDA to explore the following two datasets

```
url_1 = "https://raw.githubusercontent.com/Carl-McBride-Ellis/datasets_public/refs/heads/main/genome_biology_FEMALE.csv"
genome_biology_FEMALE = pd.read_csv(url_1)
url_2 = "https://raw.githubusercontent.com/Carl-McBride-Ellis/datasets_public/refs/heads/main/genome_biology_MALE.csv"
genome_biology_MALE = pd.read_csv(url_2)
```

Practical session II

Apply the various EDA techniques to the competition dataset.

Some example notebooks

EDA

- MSCI EDA which makes sense
- Stargate SG1 Dialogue Text Analysis
- Birdcall Recognition: EDA and Audio FE
- Jane Street: EDA of day 0 and feature importance

Data analytics

- How are the Ladies and the Gents doing?
- Head in the Clouds